

Dokumentacja Implementacyjna Projektu: Generowanie Grafów Planarnych

Aleksandra Rybałko, Nella Karczewicz

31 marca 2026

1 Cel i zakres implementacji

Celem niniejszej dokumentacji jest przedstawienie szczegółów technicznych oraz architektury programu napisanego w języku C, służącego do automatycznego rozmieszczania wierzchołków grafów planarnych. Implementacja koncentruje się na zapewnieniu stabilności obliczeń fizycznych, efektywnym zarządzaniu pamięcią oraz rygorystycznej walidacji danych wejściowych.

2 Architektura systemu i struktury danych

Aplikacja została zaprojektowana w oparciu o paradygmat programowania modularnego. Pozwala to na ścisłą separację logiki wczytywania danych, zarządzania strukturami oraz algorytmów rozmieszczania wierzchołków.

2.1 Podział na moduły funkcjonalne

Kod źródłowy został podzielony na niezależne jednostki logiczne, co ułatwia konserwację i niezależne testowanie komponentów:

- **graph.h**: Centralny moduł definicji struktur. Zawiera opisy typów `Node`, `Edge` oraz `Graph`, stanowiące wspólny interfejs dla całego systemu.
- **io.c / io.h**: Moduł wejścia/wyjścia. Odpowiada za niskopoziomową obsługę plików, dwufazową walidację składniową, sanityzację danych oraz eksport wyników (TXT/BIN).
- **layout_tutte.c / layout_tutte.h**: Implementacja deterministycznego algorytmu barycentrycznego oraz geometrycznych testów przecięć krawędzi (CCW).
- **fruchterman.c / fruchterman.h**: Implementacja heurystycznego modelu siłowego wraz z mechanizmami grawitacji i chłodzenia układu.
- **main.c**: Moduł sterujący. Zarządza logiką CLI (*Command Line Interface*), koordynuje przepływ danych i realizuje pętlę wielokrotnych prób dla algorytmów probabilistycznych.

2.2 Reprezentacja grafu w pamięci operacyjnej

Program wykorzystuje trzy kluczowe struktury zdefiniowane w pliku `graph.h`.

2.2.1 Wierzchołek (Node)

Struktura ta przechowuje pełny stan fizyczny pojedynczego punktu:

```
typedef struct {
    int id;           // Oryginalny identyfikator z pliku
    double x, y;      // Aktualne współrzędne położenia
    double dx, dy;    // Wektory sił (akumulatory przesunięcia)
} Node;
```

Listing 1: Struktura wierzchołka w `graph.h`

Analiza: Zastosowanie pól `dx` i `dy` wewnątrz struktury pozwala na uniknięcie dodatkowych alokacji tablic pomocniczych podczas iteracji algorytmu Fruchtermanna, co znacząco poprawia wydajność procesora.

2.2.2 Krawędź (Edge)

Krawędzie definiują relacje sąsiedztwa w grafie:

```
typedef struct {
    int u_idx;        // Indeks wierzchołka A w głównej tablicy
    int v_idx;        // Indeks wierzchołka B w głównej tablicy
    double weight;     // Waga połączenia
} Edge;
```

Listing 2: Struktura krawędzi w `graph.h`

Analiza: Przechowywanie indeksów (`u_idx`, `v_idx`) zamiast surowych identyfikatorów pozwala na dostęp do danych sąsiadów w czasie stałym $O(1)$, co jest niezbędne dla szybkości działania metody barycentrycznej.

2.2.3 Główny kontener (Graph)

Agreguje wszystkie zasoby grafu w jednym obiekcie:

```
typedef struct {
    Node *nodes;      // Dynamiczna tablica wierzchołków
    Edge *edges;      // Dynamiczna tablica krawędzi
    int node_count;    // Liczba wczytanych wierzchołków
    int edge_count;    // Liczba wczytanych krawędzi
    double width;      // Szerokość obszaru (domyślnie 1000.0)
    double height;     // Wysokość obszaru (domyślnie 1000.0)
} Graph;
```

2.3 Schemat przepływu danych i cykl życia programu

Proces przetwarzania danych jest zorganizowany w sposób kaskadowy, z wbudowanymi punktami kontrolnymi (tzw. "strażnikami"), które mogą przerwać wykonanie w przypadku wykrycia błędów krytycznych.

1. **Inicjalizacja CLI:** Moduł `main.c` przetwarza argumenty użytkownika, określając ścieżki plików oraz wybór algorytmu.
2. **Walidacja Diagnostyczna:** Moduł `io.c` skanuje plik wejściowy bez alokacji struktur, raportując ewentualne błędy formatu w konkretnych liniach.
3. **Budowa Topologii:** Po sukcesie walidacji następuje dynamiczna alokacja pamięci w strukturze `Graph` i mapowanie identyfikatorów zewnętrznych na indeksy tablicy.
4. **Weryfikacja Topologiczna:** Program sprawdza spójność grafu (DFS) oraz potencjalną planarność (wzór Eulera).
5. **Obliczenia Geometryczne:** Wybrany moduł (`Tutte` lub `Fruchterman`) modyfikuje współrzędne `x`, `y` w strukturach `Node`.
6. **Eksport i Sprzątanie:** Wyniki są zapisywane do pliku, a pamięć dynamiczna jest zwalniana przez funkcję `free_graph`.

3 Zarządzanie pamięcią i cykl życia obiektu

Program realizuje model bezpiecznego zarządzania pamięcią, dostosowując rozmiar struktur do rzeczywistej wielkości grafu.

3.1 Alokacja dynamiczna

Pamięć jest rezerwowana wewnątrz modułu `io.c` po zakończeniu fazy walidacji pliku:

```
g->edges = malloc(1000 * sizeof(Edge));
g->nodes = calloc(1000, sizeof(Node));
```

Listing 3: Alokacja w funkcji `read_graph`

Analiza: Wykorzystanie `calloc` gwarantuje domyślną inicjalizację pól zerem, co zapobiega błędom "garbage values" przy startowaniu symulacji fizycznej.

3.2 Zwalnianie zasobów

Za czyszczenie pamięci odpowiada funkcja `free_graph`, wywoływana w każdym scenariuszu zakończenia pracy programu:

```
void free_graph(Graph *g) {
    if(g->nodes) free(g->nodes);
    if(g->edges) free(g->edges);
    g->nodes = NULL;
    g->edges = NULL;
}
```

4 Moduł wejścia/wyjścia i rygorystyczna walidacja danych

Moduł `io.c` stanowi fundament stabilności aplikacji. Jego zadaniem jest nie tylko odczyt danych, ale przede wszystkim ich filtracja, ujednolicenie oraz weryfikacja założeń topologicznych grafu przed etapem obliczeniowym.

4.1 Automatyczna detekcja formatu i separatorów

Program został wyposażony w inteligentny mechanizm rozpoznawania struktury pliku, co pozwala na obsługę różnych standardów zapisu danych (CSV, SSV).

```
static char detect_separator(const char *line) {
    return (strchr(line, ';' ) == NULL && strchr(line, ',' ) != NULL)
        ? ',' : ';';
}
```

Listing 4: Funkcja `detect_separator`

Wyjaśnienie logiki: Funkcja analizuje surowy ciąg znaków w poszukiwaniu separatorów. Priorytetyzacja średnika nad przecinkiem zapobiega błędnej interpretacji w przypadku, gdy przecinek pełni rolę separatora dziesiętnego w wagach krawędzi.

4.2 Sanityzacja danych i obsługa błędów lokalizacji

Ze względu na różnice w zapisie liczb zmiennoprzecinkowych (kropka vs przecinek), zaimplementowano funkcję czyszczącą.

```
int sanitize(char *s) {
    int changed = 0;
    for (int i = 0; s[i]; i++) {
        if (s[i] == ',') {
            s[i] = '.';
            changed = 1;
        }
    }
    return changed;
}
```

Listing 5: Funkcja `sanitize` w `io.c`

Wyjaśnienie logiki: Funkcja iteruje po napisie i zamienia każdy przecinek na kropkę. Jest to kluczowe dla poprawnego działania funkcji `sscanf`, która w standardzie C oczekuje kropki jako separatora dziesiętnego.

4.3 Zapis danych w formacie binarnym i tekstowym

Funkcja `save_graph` pozwala na eksport współrzędnych w zależności od potrzeb użytkownika.

```
if (strcmp(format, "bin") == 0) {
    fwrite(&(g->node_count), sizeof(int), 1, f);
}
```

```
fwrite(g->nodes, sizeof(Node), g->node_count, f);
}
```

Listing 6: Obsługa formatu binarnego w save_graph

Wyjaśnienie logiki: W trybie binarnym dane są zapisywane bezpośrednio z pamięci RAM przy użyciu `fwrite`. Pozwala to na uniknięcie błędów zaokrągleń oraz drastycznie zmniejsza rozmiar pliku wyjściowego w porównaniu do zapisu tekstowego `fprintf`.

5 Założenia teoretyczne i walidacja topologiczna

Aby algorytmy rozmieszczania wierzchołków mogły wygenerować poprawny i czytelny rysunek, graf wejściowy musi spełniać określone założenia matematyczne. Program weryfikuje te warunki na etapie walidacji wstępnej, niezależnie od wybranego algorytmu.

5.1 Fundamenty teoretyczne: Struktura grafu, Spójność i Planarność

W celu zachowania stabilności obliczeń numerycznych oraz poprawności topologicznej, system operuje na tzw. **grafach prostych** i sprawdza ich kluczowe parametry:

- **Wymóg grafu prostego:** Program automatycznie odrzuca **pętle własne** (połączenie wierzchołka z samym sobą) oraz ignoruje **krawędzie wielokrotne** (duplikaty między tymi samymi węzłami). Pętle własne uniemożliwiłyby działanie modelu siłowego ze względu na zerową odległość (dzielenie przez zero), a duplikaty zaburzałyby statyczną weryfikację planarności.
- **Spójność (Connectivity):** Jest to warunek konieczny dla algorytmów siłowych i barycentrycznych. W grafie spójnym istnieje ścieżka między każdą parą wierzchołków. Brak spójności spowodowałby "nakładanie się" osobnych komponentów na siebie lub ich niekontrolowane rozbieganie się w obszarze roboczym pod wpływem sił odpychania.
- **Planarność i wzór Eulera:** Graf jest planarny, jeśli można go narysować bez przecięć krawędzi. Wykorzystujemy tu wniosek z twierdzenia Eulera: dla grafu planarnego o $V \geq 3$ wierzchołkach, liczba krawędzi E musi spełniać warunek $E \leq 3V - 6$. Jest to pierwszy "strażnik" blokujący przetwarzanie zbyt gęstych danych.

5.2 Weryfikacja spójności grafu (Algorytm DFS)

Zaimplementowano funkcję wykorzystującą przeszukiwanie w głąb (DFS) ze stosem, która zlicza odwiedzone węzły startując od pierwszego wczytanego punktu.

```
int is_graph_connected(Graph *g) {
    if (g->node_count <= 1) return 1;
    int *visited = calloc(g->node_count, sizeof(int));
    int *stack = malloc(g->node_count * sizeof(int));
    int top = -1;

    stack[++top] = 0; // Start od pierwszego wierzchołka
    visited[0] = 1;
```

```

int visited_count = 1;

while (top >= 0) {
    int u = stack[top--];
    for (int i = 0; i < g->edge_count; i++) {
        int v = -1;
        if (g->edges[i].u_idx == u) v = g->edges[i].v_idx;
        else if (g->edges[i].v_idx == u) v = g->edges[i].u_idx;

        if (v != -1 && !visited[v]) {
            visited[v] = 1;
            stack[++top] = v;
            visited_count++;
        }
    }
}
free(visited); free(stack);
return (visited_count == g->node_count);
}

```

Listing 7: Funkcja `is_graph_connected`

Logika blokady: Jeśli $visited_count < g->node_count$, graf jest niespójny. Program przerywa pracę z kodem błędu 9, co zapobiega generowaniu błędnych wyników.

5.3 Weryfikacja potencjalnej planarności (Wzór Eulera)

Drugim etapem walidacji wspólnej jest statyczne sprawdzenie gęstości grafu przed uruchomieniem kosztownych obliczeń fizycznych.

```

int is_potentially_planar(Graph *g) {
    if (g->node_count <= 2) return 1;
    return (g->edge_count <= (3 * g->node_count - 6));
}

```

Listing 8: Funkcja `is_potentially_planar`

Logika blokady: Jeśli warunek nie zostanie spełniony, program informuje o braku potencjalnej planarności (kod błędu 7). Stanowi to filtr chroniący przed próbą siłowego rozmieszczania struktur, które matematycznie nie posiadają reprezentacji planarnej.

6 Algorytm Tutte’go – Deterministyczna Metoda Barycentryczna

Algorytm Tutte’go, znany również jako metoda barycentryczna, stanowi fundament deterministycznego podejścia do wizualizacji grafów planarnych. W przeciwieństwie do metod heurystycznych, algorytm ten opiera się na rozwiązaniu układu równań liniowych, co gwarantuje uzyskanie unikalnego i stabilnego wyniku dla danej topologii grafu.

6.1 Logika i podstawy matematyczne

Głównym założeniem algorytmu jest fakt, że każdy wierzchołek wewnętrzny powinien znajdować się w tzw. barycentrum (środku ciężkości) swoich sąsiadów.

Zgodnie z twierdzeniem Tutte’a, aby uzyskać ułożenie planarne (brak przecięć krawędzi), muszą zostać spełnione trzy warunki:

1. Graf musi być planarny i trójspójny.
2. Wierzchołki zewnętrznej ściany grafu (rama) muszą tworzyć wielokąt wypukły.
3. Każdy wierzchołek wewnętrzny musi być połączony z co najmniej trzema innymi wierzchołkami.

Pozycja każdego ruchomego wierzchołka v_i jest wyznaczana jako średnia arytmetyczna pozycji jego sąsiadów:

$$x_i = \frac{1}{|Adj(v_i)|} \sum_{j \in Adj(v_i)} x_j, \quad y_i = \frac{1}{|Adj(v_i)|} \sum_{j \in Adj(v_i)} y_j$$

6.2 Implementacja procesu inicjalizacji

Pierwszym etapem algorytmu jest zdefiniowanie "kotwic" układu, czyli wierzchołków stałych, które nie podlegają procesowi relaksacji.

```
void initialize_tutte_fixed_nodes(Graph *g) {
    if (g->node_count < 3) return;

    double centerX = g->width / 2.0;
    double centerY = g->height / 2.0;
    // Promień ramy dostosowany do wymiarów obszaru roboczego
    double radius = (g->width < g->height ? g->width : g->height) / 3.0;

    // Rozmieszczenie pierwszych 4 wierzchołków na obwodzie koła
    for (int i = 0; i < 4 && i < g->node_count; i++) {
        double angle = 2.0 * M_PI * i / 4.0;
        g->nodes[i].x = centerX + radius * cos(angle);
        g->nodes[i].y = centerY + radius * sin(angle);
    }
}
```

Listing 9: Inicjalizacja ramy grafu w layout_tutte.c

Wyjaśnienie logiki: Program rezerwuje indeksy 0 – 3 jako wierzchołki zewnętrzne. Wykorzystanie funkcji `cos` i `sin` zapewnia idealnie symetryczne, wypukłe ułożenie startowe, co jest warunkiem koniecznym dla powodzenia algorytmu.

6.3 Iteracyjne rozwiązywanie układu (Metoda relaksacji)

Zamiast budować złożone macierze rzadkie, w projekcie zastosowano metodę relaksacji, która w sposób przybliżony dąży do stanu równowagi.

```
for (int iter = 0; iter < 200; iter++) {
    // Aktualizujemy tylko wierzchołki ruchome (od indeksu 4 wzwyż)
    for (int i = 4; i < g->node_count; i++) {
        double sum_x = 0, sum_y = 0;
        int neighbors_count = 0;

        // Przeszukiwanie listy krawędzi w celu znalezienia sąsiadów
        // w wierzchołku i
        for (int j = 0; j < g->edge_count; j++) {
            int neighbor_idx = -1;
            if (g->edges[j].u_idx == i) neighbor_idx = g->edges[j].v_idx;
            else if (g->edges[j].v_idx == i) neighbor_idx = g->edges[j].u_idx;

            if (neighbor_idx != -1) {
                sum_x += g->nodes[neighbor_idx].x;
                sum_y += g->nodes[neighbor_idx].y;
                neighbors_count++;
            }
        }
        // Przesunięcie wierzchołku do barycentrum jego sąsiedztwa
        if (neighbors_count > 0) {
            g->nodes[i].x = sum_x / neighbors_count;
            g->nodes[i].y = sum_y / neighbors_count;
        }
    }
}
```

Listing 10: Logika obliczania środka ciężkości

Wyjaśnienie logiki: Pętla zewnętrzna wykonuje 200 iteracji, co empirycznie dobrano jako wartość zapewniającą stabilizację wizualną grafu. Wewnątrz, dla każdego węzła ruchomego, program sumuje pozycje sąsiadów i wyciąga średnią. Jeśli wierzchołek nie posiada sąsiadów (`neighbors_count == 0`), pozostaje w miejscu, co zapobiega błędowi dzielenia przez zero.

7 Algorytm Fruchterman-Reingold – Heurystyka Siłowa

Algorytm Fruchterman-Reingold opiera się na analogii fizycznej, w której wierzchołki traktowane są jak cząsteczki atomowe, a krawędzie jak sprężyny.

7.1 Logika oddziaływań fizycznych

Głównym celem algorytmu jest znalezienie konfiguracji o najniższej energii. Kluczowym parametrem jest stała optymalnej odległości k :

$$k = \sqrt{\frac{Width \cdot Height}{|V|}}$$

7.2 Specyfika implementacji i dążenie do planarności

W klasycznym ujęciu algorytm ten jest heurystyką estetycznego rozmieszczania, która dąży do równomiernego rozłożenia węzłów, ale nie gwarantuje uzyskania układu planarnego. W niniejszym projekcie algorytm został jednak rozbudowany o mechanizmy wymuszające bezkolizyjność:

- **Zastosowanie grawitacji:** Dodanie wektora siły dociągającej do środka zapobiega nadmiernemu rozproszeniu wierzchołków.
- **Wielokrotne losowe starty:** Zaimplementowano pętlę *Retry Logic* (do 3000 prób), co pozwala na uniknięcie lokalnych minimów energetycznych.
- **Weryfikacja testem przecięć:** Program uznaje działanie za zakończone sukcesem dopiero po uzyskaniu statusu `is_layout_planar == 1`.

7.3 Implementacja sił odpychania i grawitacji

W przeciwieństwie do sił przyciągania, siły odpychania działają na każdą parę wierzchołków w grafie, co nadaje algorytmowi złożoność $O(V^2)$.

```
// 1. ODPYCHANIE: Każdy wierzchołek odpycha się od każdego innego
for (int i = 0; i < graph->node_count; i++) {
    graph->nodes[i].dx = 0; // Resetowanie wektora przesunięcia
    graph->nodes[i].dy = 0;

    // Grawitacja: słaba siła dociągająca wierzchołek do środka
    // ekranu
    double dx_center = centerX - graph->nodes[i].x;
    double dy_center = centerY - graph->nodes[i].y;
    graph->nodes[i].dx += dx_center * 0.05;
    graph->nodes[i].dy += dy_center * 0.05;

    for (int j = 0; j < graph->node_count; j++) {
        if (i == j) continue;
        double vx = graph->nodes[i].x - graph->nodes[j].x;
        double vy = graph->nodes[i].y - graph->nodes[j].y;
        double dist = sqrt(vx * vx + vy * vy);

        if (dist < 0.01) dist = 0.01; // Unikanie osobliwości przy
        // dist=0

        double force = (k * k) / dist; // f_r(d) = k^2 / d
```

```

        graph->nodes[i].dx += (vx / dist) * force;
        graph->nodes[i].dy += (vy / dist) * force;
    }
}

```

Listing 11: Obliczanie sumarycznych wektorów sił

Wyjaśnienie logiki: Wektory dx i dy pełnią rolę akumulatorów sił. Dodanie grawitacji (siła dociągająca do środka) zapobiega "rozlatywaniu się" niespójnych komponentów grafu poza obszar roboczy.

7.4 Mechanizm chłodzenia (Simulated Annealing)

Aby układ mógł "zastygnąć" w stabilnej pozycji, wprowadzono parametr temperatury t , który ogranicza maksymalne przemieszczenie wierzchołka w jednej iteracji.

```

for (int i = 0; i < graph->node_count; i++) {
    double disp_dist = sqrt(graph->nodes[i].dx * graph->nodes[i].dx
        +
        graph->nodes[i].dy * graph->nodes[i].dy)
        ;

    if (disp_dist > 0) {
        // Ograniczenie przesunięcia aktualną temperaturą t
        double limited_dist = (disp_dist < t) ? disp_dist : t;
        graph->nodes[i].x += (graph->nodes[i].dx / disp_dist) *
            limited_dist;
        graph->nodes[i].y += (graph->nodes[i].dy / disp_dist) *
            limited_dist;
    }
    // (...) ograniczenie do ramki obszaru
}
t *= 0.95; // Stopniowe chłodzenie systemu o 5% w każdym kroku

```

Listing 12: Aktualizacja pozycji i chłodzenie

Wyjaśnienie logiki: Temperatura początkowa jest wysoka, co pozwala wierzchołkom na duże skoki i wyjście z lokalnych minimów energetycznych. Z każdą iteracją temperatura maleje ($t *= 0.95$), co powoduje, że graf staje się coraz bardziej "sztywny" i stabilny.

8 Scenariusze testowe i weryfikacja wyników

W celu potwierdzenia niezawodności aplikacji przeprowadzono serię testów systemowych. Dokumentacja zestawia fragmenty plików wejściowych z logami z terminala, co pozwala na precyzyjną weryfikację logiki walidacyjnej modułu `io.c`.

8.1 Szczegółowa analiza przypadków testowych (Pliki `.txt`)

8.1.1 Błędny format danych (`err_litera.txt`)

Test sprawdza odporność walidatora na znaki niebędące cyframi w polach numerycznych (`U`, `V`, `Waga`).

Fragment struktury:

```
krawedz1;1;2x;1.0
krawedz2;2;X;1.5
krawedz3;3;4;0.8
```

```
=====
      RAPORT ANALIZY DANYCH WEJSCIOWYCH
=====
1      | [!!! BŁĄD KRYTYCZNY !!!] Wykryto litery lub zły format.
2      | [!!! BŁĄD KRYTYCZNY !!!] Wykryto litery lub zły format.
3      | Krawedz krawedz3: 3 -> 4 | OK

STOP! Znaleziono 2 błędów krytycznych w strukturze pliku.
-> INFO: Poprawny format to: Nazwa;U;V;Waga
-> Separator danych to ŚREDNIK (;), a separator dziesiętny to
    KROPKA (.)
```

Listing 13: Log terminala - wykrycie liter

8.1.2 Niespójność grafu (err_niespojny.txt)

Weryfikacja algorytmu DFS. Program musi wykryć, że graf składa się z kilku niepołączonych komponentów.

Fragment struktury:

```
k1;1;2;1.0
k2;2;3;1.0
k3;4;5;1.0
```

```
=====
      RAPORT ANALIZY DANYCH WEJSCIOWYCH
=====
1      | Krawedz k1: 1 -> 2 | OK
2      | Krawedz k2: 2 -> 3 | OK
3      | Krawedz k3: 4 -> 5 | OK
Uruchamiam przetwarzanie...

Wczytywanie zakończone sukcesem.
Statystyki: 5 węzłów, 3 krawędzie.
Uruchamiam algorytm spójności (DFS) dla powyższych danych...

Sprawdzanie spójności grafu...

=====
      BŁĄD KRYTYCZNY: GRAF NIESPÓJNY
=====
Dane z pliku wejściowego [err_niespojny.txt] są błędne:
-> Graf składa się z kilku niepołączonych ze sobą części.
-> Algorytm wymaga grafu spójnego, aby poprawnie wyznaczyć pozycje.
=====
```

Listing 14: Log terminala - graf niespójny

8.1.3 Brak planarności – zbyt duża gęstość (err_gestosc.txt)

Test sprawdzający warunek Eulera ($E \leq 3V - 6$). Program odrzuca grafy, których matematycznie nie da się narysować bez przecięć.

```
=====
      RAPORT ANALIZY DANYCH WEJSCIOWYCH
=====
1      | Krawedz k1: 1 -> 2 | OK
2      | Krawedz k2: 1 -> 3 | OK
3      | Krawedz k3: 1 -> 4 | OK
4      | Krawedz k4: 1 -> 5 | OK
5      | Krawedz k5: 2 -> 3 | OK
6      | Krawedz k6: 2 -> 4 | OK
7      | Krawedz k7: 2 -> 5 | OK
8      | Krawedz k8: 3 -> 4 | OK
9      | Krawedz k9: 3 -> 5 | OK
10     | Krawedz k10: 4 -> 5 | OK
Uruchamiam przetwarzanie...

Wczytywanie zakończone sukcesem.
Statystyki: 5 węzłów, 10 krawędzi.
Uruchamiam algorytm spójności (DFS) dla powyższych danych...

Sprawdzanie spójności grafu...
Sukces: Graf jest spójny. Przechodzę do obliczeń.
[INFO] Graf jest nieplanarny (E > 3V-6). Jeśli chcesz, aby program
       wygenerował graf planarny, to uruchom go ponownie.
```

Listing 15: Log terminala - przekroczenie limitu krawędzi

8.1.4 Minimalna liczba wierzchołków (err_zamaly.txt)

Algorytm Tutte'a wymaga minimum 3 węzłów do stworzenia ramy. Test sprawdza blokadę dla zbyt małych struktur.

```
=====
      RAPORT ANALIZY DANYCH WEJSCIOWYCH
=====
1      | Krawedz k1: 1 -> 2 | OK
Uruchamiam przetwarzanie...

Wczytywanie zakończone sukcesem.
Statystyki: 2 węzłów, 1 krawędzi.
Uruchamiam algorytm spójności (DFS) dla powyższych danych...

=====
      BŁĄD: ZA MAŁĄ LICZBA WIERZCHOŁKÓW
=====
Napotkano 2 wierzchołków.
-> Potrzeba >= 3 wierzchołków, by wygenerować graf planarny.
=====
```

Listing 16: Log terminala - za mały graf

8.1.5 Autokorekta separatora dziesiętnego (test_autokorekta.txt)

Weryfikacja funkcji `sanitize`, która zamienia przecinki na kropki, umożliwiając poprawny odczyt wag przez `sscanf`.

Fragment struktury:

```
krawedz1;1;2;1,50
krawedz2;2;3;2,75
krawedz3;3;1;0,99
```

```
=====
      RAPORT ANALIZY DANYCH WEJSCIOWYCH
=====
1      | Krawedz krawedz1: 1 -> 2 | OK
2      | Krawedz krawedz2: 2 -> 3 | OK
3      | Krawedz krawedz3: 3 -> 1 | OK
INFO: Dokonano 3 autokorekt separatora dziesiętnego.
Uruchamiam przetwarzanie...

Wczytywanie zakończone sukcesem.
Statystyki: 3 węzłów, 3 krawędzi.
Uruchamiam algorytm spójności (DFS) dla powyższych danych...

Sprawdzanie spójności grafu...
Sukces: Graf jest spójny. Przechodzę do obliczeń.
Uruchamiam symulację Fruchtermana (max 3000 prób)...

>>> SUKCES: Wynik został zapisany do pliku [out.txt] <<<
>>> FORMAT: Plik TEKSTOWY (.txt) <<<
```

Listing 17: Log terminala - INFO: Dokonano autokorekt

8.1.6 Obsługa pustych linii i białych znaków (test_puste.txt)

Parser musi ignorować wiersze niezawierające danych, aby nie przerywać procesu wczytywania grafu. **Fragment struktury:**

```
k1;1;2;1.0

k2;2;3;1.5

k3;3;1;0.8
```

```
=====
      RAPORT ANALIZY DANYCH WEJSCIOWYCH
=====
1      | Krawedz k1: 1 -> 2 | OK
```

```

2      | [INFO] Pominięto pustą linię.
3      | Krawedz k2: 2 -> 3 | OK
4      | [INFO] Pominięto pustą linię.
5      | Krawedz k3: 3 -> 1 | OK
Uruchamiam przetwarzanie...
Linia 2: [INFO] Napotkano pustą linię - pomijam.
Linia 4: [INFO] Napotkano pustą linię - pomijam.

Wczytywanie zakończone sukcesem.
Statystyki: 3 węzłów, 3 krawędzi.
Uruchamiam algorytm spójności (DFS) dla powyższych danych...

Sprawdzanie spójności grafu...
Sukces: Graf jest spójny. Przechodzę do obliczeń.
Uruchamiam symulację Fruchtermana (max 3000 prób)...

>>> SUKCES: Wynik został zapisany do pliku [out.txt] <<<
>>> FORMAT: Plik TEKSTOWY (.txt) <<<

```

Listing 18: Log terminala - pomijanie pustych linii

8.2 Pozostałe zweryfikowane mechanizmy

Poza testami opartymi na plikach tekstowych, aplikacja została poddana weryfikacji funkcjonalnej w następujących obszarach:

- **Parametry CLI:** Sprawdzono reakcję na brak flag `-i`, `-o` oraz wpisanie nieistniejących algorytmów. Program każdorazowo przerywa pracę i wyświetla instrukcję obsługi `Usage`.
- **Logika struktur danych:** Przetestowano mechanizm odrzucania duplikatów krawędzi oraz ignorowania pętli własnych, co zapewnia stabilność obliczeniową.
- **Formaty wyjściowe:** Zweryfikowano poprawność zapisu binarnego (`.bin`) oraz tekstowego przy użyciu funkcji `save_graph`.
- **Bezpieczeństwo I/O:** Program poprawnie identyfikuje brak pliku wejściowego (Kod 2) oraz brak uprawnień do zapisu.

8.3 Test pozytywny – wzorcowy przebieg wybranego algorytmu: Tutte’a

W celu weryfikacji pełnej ścieżki krytycznej programu przeprowadzono test na poprawnym, spójnym grafie planarnym. Wykorzystano algorytm Tutte’a z domyślnym zapisem do pliku tekstowego.

Zawartość pliku wejściowego (poprawny_plik.txt):

```

K1;1;2;1.0
K2;2;3;1.0
K3;3;1;1.0

```

8.3.1 Logi z konsoli (Przebieg operacji)

Poniższy listing przedstawia pełny raport walidacyjny oraz potwierdzenie zakończenia obliczeń przez silnik graficzny.

```
=====
      RAPORT ANALIZY DANYCH WEJSCIOWYCH
=====
1          | Krawedz K1: 1 -> 2 | OK
2          | Krawedz K2: 2 -> 3 | OK
3          | Krawedz K3: 3 -> 1 | OK
Uruchamiam przetwarzanie...

Wczytywanie zakończone sukcesem.
Statystyki: 3 węzłów, 3 krawędzi.
Uruchamiam algorytm spójności (DFS) dla powyższych danych...

Sprawdzanie spójności grafu...
Sukces: Graf jest spójny. Przechodzę do obliczeń.
Sukces! Graf planarny (Tutte).

>>> SUKCES: Wynik został zapisany do pliku [out.txt] <<<
>>> FORMAT: Plik TEKSTOWY (.txt) <<<
```

Listing 19: Kompletny przebieg programu dla danych poprawnych

8.3.2 Weryfikacja struktury pliku wynikowego

Poniżej przedstawiono zawartość wygenerowanego pliku `out.txt`. Wierzchołki zostały rozłożone na planie trójkąta równobocznego (standardowa rama algorytmu Tutte'a).

```
1 833.33 500.00
2 500.00 833.33
3 166.67 500.00
```

Listing 20: Zawartość pliku `out.txt`

Wnioski z testu: Program poprawnie zidentyfikował strukturę grafu, przeszedł walidację DFS i wygenerował plik tekstowy o poprawnej strukturze (ID, X, Y), co potwierdza pełną sprawność modułów obliczeniowych i eksportujących.

8.4 Wnioski z testów

Przeprowadzone testy potwierdzają, że program jest odporny na błędy użytkownika oraz nieprawidłowe formatowanie danych. Wielowarstwowa walidacja (od znaków ASCII po spójność topologiczną) gwarantuje, że algorytmy pracują na poprawnych danych.